



UNIVERSITÀ
degli STUDI
di CATANIA

RELAZIONE DEL PROGETTO DI MACHINE LEARNING

ELIA ANTONIO MALVÈ

CDLM PHYSICS LM17

Sommario

Problema	2
Dataset	2
Dataset di training	2
Dataset di test	3
Etichettatura dei dati	4
Metodi	4
Algoritmo di learning	4
Architettura della CNN	4
Valutazione	5
Esperimenti	5
Scelta del modello	5
Scelta degli iperparametri	5
Demo	9
Codice	10
Conclusioni	10

Problema

Il progetto ha come obiettivo lo sviluppo di un modello che consenta di calcolare il numero di persone presenti in un'immagine (o crowd counting) basato su una rete neurale convoluzionale (CNN). L'algoritmo viene allenato su un dataset liberamente scaricabile presentante immagini di folla, ciascuna marcata con il numero di persone presenti. Viene poi testato su un dataset prodotto in proprio. L'obiettivo è stimare il numero di individui in scenari realistici, mediante regressione. Alcuni possibili utilizzi di tale software potrebbero essere legati al calcolo ed ottimizzazione del numero di persone in un centro commerciale, al fine di massimizzare i guadagni, piuttosto che al controllo di un'area chiusa per ragioni di sicurezza.

Dataset

Dataset di training

Il dataset utilizzato ([accessibile qui](#)), è stato costruito a partire da risorse pubbliche disponibili online, nella fattispecie da Kaggle, un sito che rappresenta un riferimento nel settore della data science. Esso contiene 2000 immagini RGB a risoluzione 480x640 pixel, le quali mostrano diversi frame di una stessa telecamera in un centro commerciale, al variare del traffico pedonale presente. Insieme alle immagini sono inclusi nel dataset i label per ogni immagine, recanti il numero di persone effettivamente presenti.



Immagine di esempio del dataset di train.

Ogni immagine è stata preprocessata (resize a 224x224 pixel) e normalizzata secondo i parametri standard di ImageNet. Sono state inoltre applicate tecniche di data augmentation (rotazioni, flip, zoom, color jitter) al fine di migliorare la generalizzazione del modello.

Dataset di test

Il dataset di test è stato costruito selezionando un insieme di immagini indipendenti, non utilizzate in fase di training né di validation, per avere una valutazione realistica della capacità del modello di generalizzare. Si sono unite diverse immagini, alcune delle quali liberamente scaricabili da internet, altre invece prodotte puntualmente allo scopo. L'etichetta è insita nel formato del nome, del tipo "test_####_%%%", con #### un id incrementale che identifica univocamente l'immagine, ed %%%% il label, ovvero il numero di persone effettivamente presenti. In totale sono state usate 50 immagini per test, di cui 30 estratte da internet e 20 prodotte dall'autore appositamente allo scopo (con l'aiuto di diversi volontari). Tutte sono state scelte in condizioni di luce ed inquadrature differenti, in modo da valutare la capacità di generalizzazione del modello. Anche su queste si è effettuato il resize a 224x224 pixel, necessario per l'input alla CNN usata, e si sono normalizzate con i parametri standard di ImageNet, ma non si è effettuato alcun tipo di data augmentation.



Immagine di esempio del dataset di test, ottenuta in proprio.

Etichettatura dei dati

Le etichette sono state fornite sotto forma di conteggi (numero di individui per immagine). Il modello è stato addestrato a predire un valore numerico, cercando di ottimizzare la distanza tra la predizione e il conteggio reale. L'etichettatura dei dati di test è stata svolta manualmente. Entrambi i dataset sono organizzati in cartelle, nello specifico la cartella "progetto_ML\dataset\train\" contiene il dataset di train, in formato .jpg ed .npz (ovvero NumPy array). Nel codice, è stato usato quest'ultimo formato per comodità. Piuttosto, il dataset di test ha percorso "progetto_ML\test_dataset\" ed è costituito da immagini in formato .jpg.

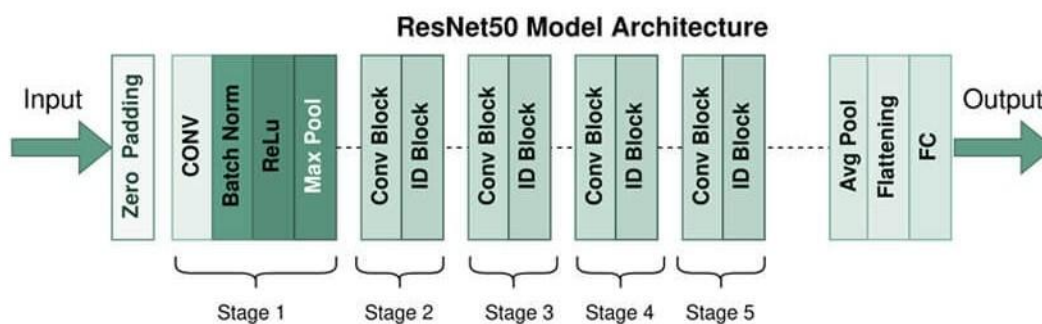
Metodi

Algoritmo di learning

Il funzionamento della CNN è basato su diverse fasi. Il primo passo svolto è il **preprocessing** delle immagini, dove si sono ridimensionate a 224x224 pixel, e normalizzate con i parametri di ImageNet, in modo da poterle dare in input ad una CNN, nello specifico ResNet50. Nel caso del dataset di train e validation, si è anche svolta data augmentation, come descritto in precedenza. È stata adottata una strategia di transfer learning, sfruttando la ResNet50 pre-addestrata come backbone. e suddividendo l'intero dataset in training set (80%) e validation set (20%). **L'ottimizzazione** è stata eseguita tramite Stochastic Gradient Descent (SGD) con momentum e parametro di regolarizzazione. L'andamento del training è inoltre visibile mediante tensorboard in tempo reale.

Architettura della CNN

La rete scelta è la ResNet50, una CNN caratterizzata da connessioni residue (l'output di un layer viene portato al livello successivo) che riducono il problema del vanishing gradient e dell'exploding gradient. All'uscita della rete, un livello fully connected è stato adattato per predire un singolo valore (il conteggio stimato delle persone). È stato applicato un dropout (spegnimento casuale di alcuni neuroni della rete, in fase di training) pari al 55% per ridurre l'overfitting.



Schema dell'architettura ResNet50.

Valutazione

Durante il training sono state monitorate le metriche Mean Absolute Error (MAE), Mean Squared Error (MSE) e Root Mean Squared Error (RMSE). In particolare, la MAE si è ritenuta l'indicatore più affidabile, poiché fisicamente sensato, in quanto errore sul conteggio. Il modello finale ed i relativi iperparametri sono stati selezionati in base alle prestazioni sul validation set.

Esperimenti

Scelta del modello

Sono stati testati diversi modelli pre-addestrati (SqueezeNet, ResNet18, ResNet34, ResNet50). Il primo tentativo è stato fatto con l'architettura SqueezeNet, in quanto particolarmente leggera e già vista durante i laboratori didattici. Ciononostante, i risultati sono stati poco soddisfacenti, pertanto si è deciso di virare verso una CNN più complessa, del tipo ResNet. Dopo qualche tentativo in tal senso, la ResNet50 ha garantito un buon compromesso tra accuratezza e costo computazionale, anche considerando la complessità delle successive (ResNet101, ResNet 152) e pertanto si è usata in tutte le successive fasi.

Scelta degli iperparametri

Sono stati effettuati diversi test, variando principalmente il learning rate, il momentum, il parametro di regolarizzazione ed il dropout. In particolare, sono stati effettuati i seguenti tentativi per identificare le migliori condizioni dei parametri:

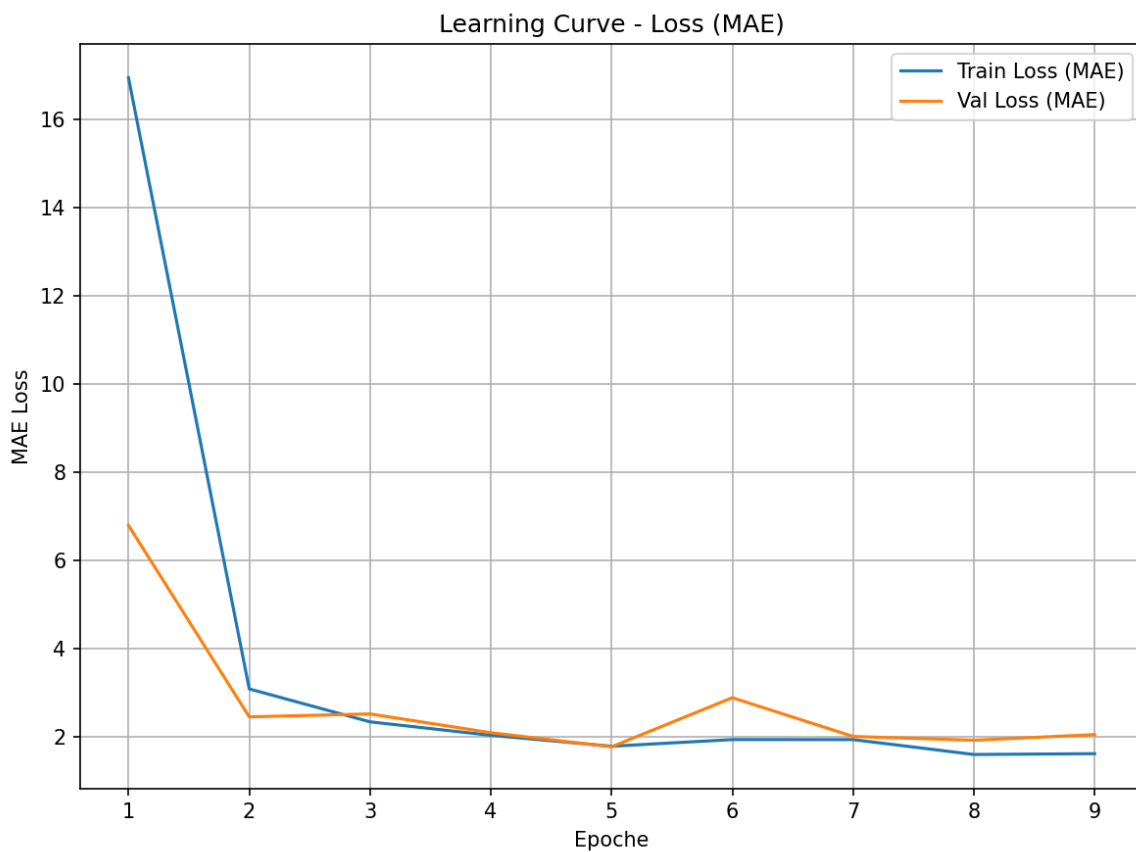
TENTATIVO	LEARNING RATE	MOMENTUM	PARAMETRO DI REGOLARIZZAZIONE	DROPOUT
1	0.001	0.95	0.0001	0.60
2	0.001	0.94	0.0001	0.57
3	0.001	0.93	0.0001	0.58
4	0.0001	0.93	0.0001	0.40
5	0.0005	0.93	0.0001	0.45
6	0.001	0.93	0.0001	0.60
7	0.001	0.92	0.0001	0.55

Avendo ottenuto i migliori parametri nella configurazione 7, si sono fissati, procedendo a variare solo uno di essi alla volta per una calibrazione fine. In particolare, si è variato il momentum nell'intervallo [0.90;0.99], ed il dropout nell'intervallo [0.50;0.60], quest'ultimo in modo da prevenire l'overfitting al meglio. Si è inoltre implementata la riduzione dinamica del learning rate, per evitare minimi locali della funzione di loss se non si ha un guadagno apprezzabile per un certo numero di epoche.

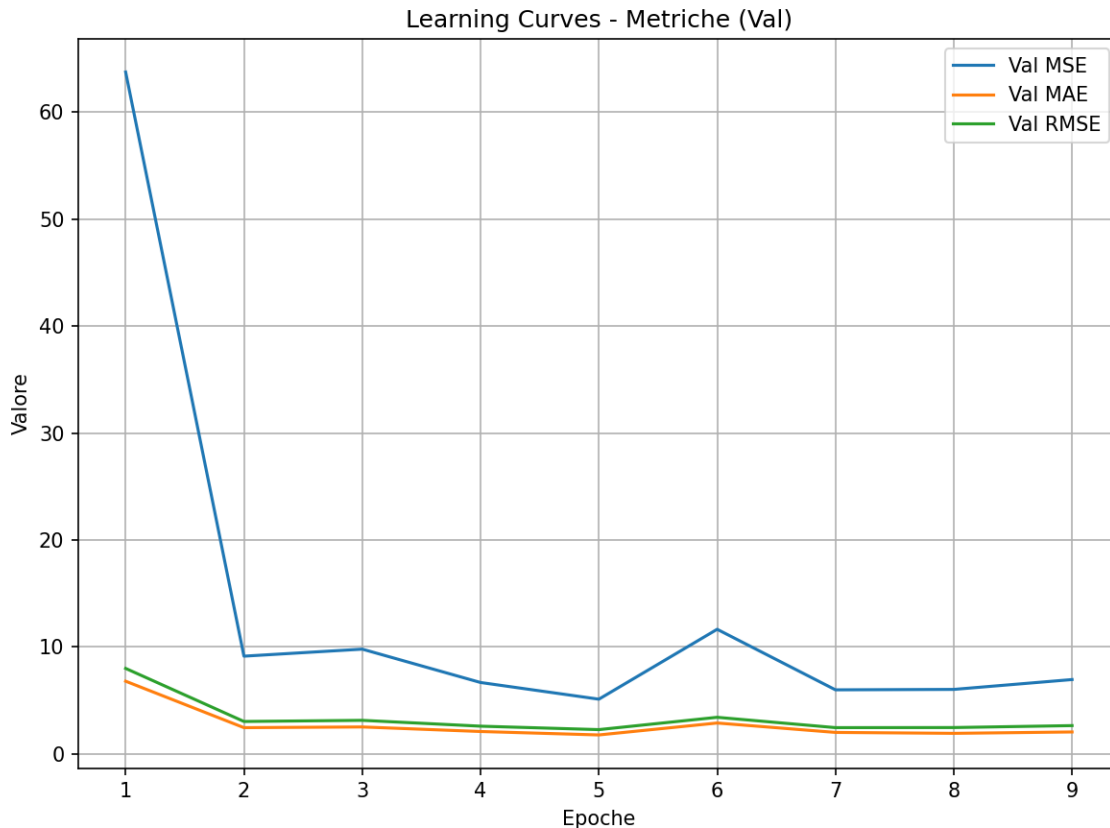
Gli iperparametri finali sono stati:

- Batch size: 32 (invariato in tutte le run, numero di campioni per SGD)
- Learning rate: 0.001
- Momentum: 0.92
- Weight decay: 0.0001
- Dropout: 0.55
- Patience: 4 epoche (epoche senza miglioramento sufficienti ad arrestare l'algoritmo)

Il training si è arrestato dopo 9 epoche per early stopping.



Curve di loss per train e test set al variare delle epoche. Nelle prime due epoche la loss di validation è più bassa di quella di train a causa del dropout, presente solo in fase di train. L'andamento è del tutto simile a quello atteso.



Andamento delle metriche al variare delle epoche.

Risultati finali sul validation set:

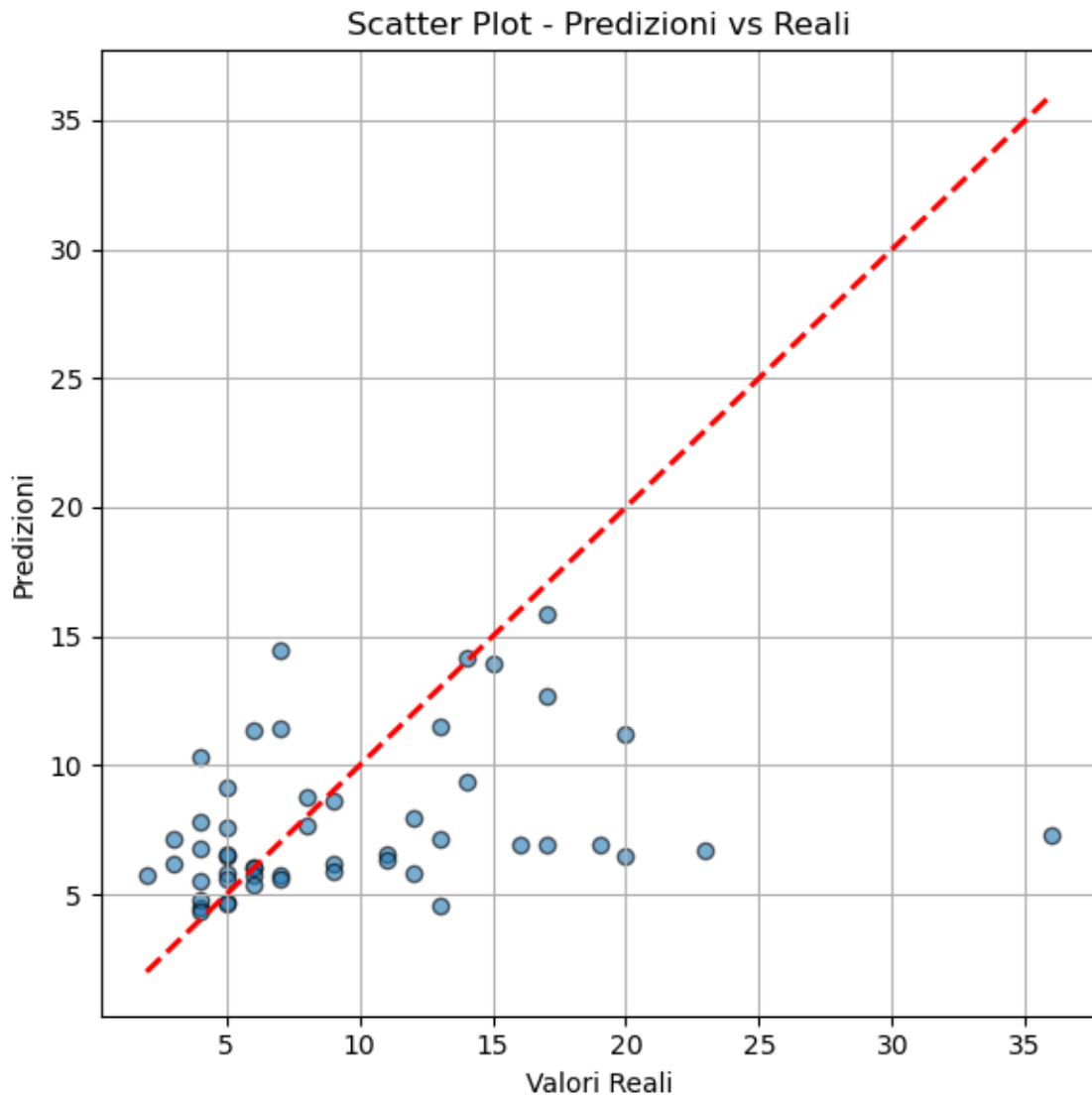
- MSE: 5.1072
- MAE: 1.7629
- RMSE: 2.2599

Usando come riferimento il MAE, il modello sbaglia il conteggio, in media, di 1.76 persone sul validation set, un risultato accettabile considerata la dimensione del dataset e la configurazione delle foto in termini di inquadratura.

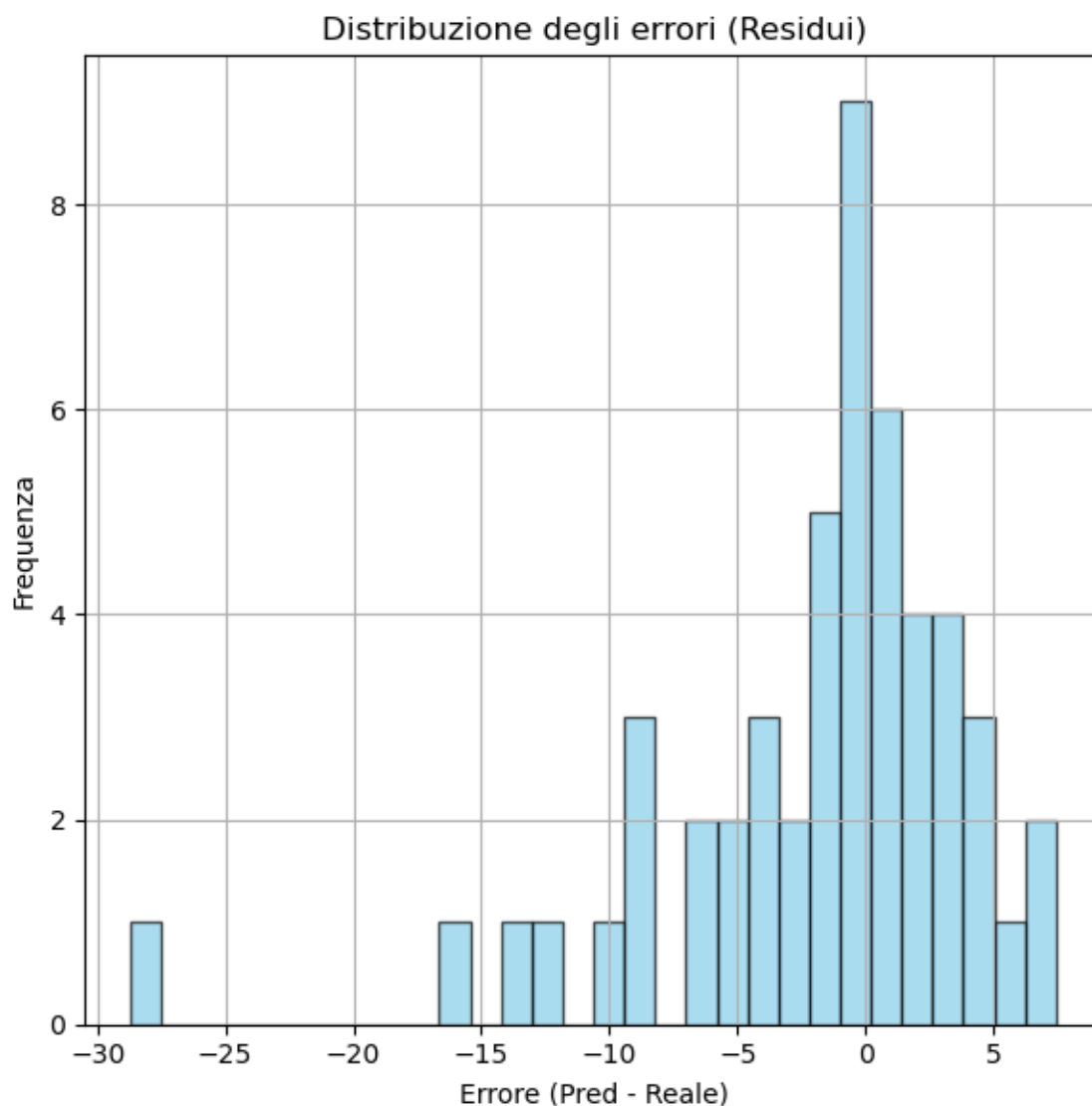
Risultati sul test set:

- MSE: 42.9007
- MAE: 4.1608
- RMSE: 6.5499
- Intervallo di confidenza MAE (95%): [2.9183, 5.5297] (Quest'ultimo parametro si è inserito poiché, a causa delle ridotte dimensioni del test set, i risultati delle metriche ottenuti possono fluttuare anche con l'aggiunta di pochi elementi. Pertanto, questo rappresenta il massimo ed il minimo MAE ottenibili, una forma di errore sulla metrica).

Le metriche ottenute sul dataset di test si discostano sensibilmente dai valori ottenuti nel training. Diversi fattori concorrono a questo fenomeno, in primis la dimensione del dataset di test piuttosto limitata, che inoltre presenta inquadrature, condizioni di luce ed ambiente piuttosto diversi rispetto al dataset usato per l'addestramento. In virtù del manifestarsi di tali condizioni, il peggioramento dei valori delle metriche non è comunque esageratamente alto, tenendo altresì in considerazione che si tratta di un modello non eccessivamente complesso.



Scatter plot recante i valori di conteggio reali con le relative predizioni. Il modello sembra effettuare meglio le predizioni con un basso numero di persone nell'immagine. I punti sulla bisettrice sono quelli che hanno lo stesso numero di predizioni e valori reali, pertanto più un punto è vicino alla retta in questione, migliore sarà stata la predizione.



Frequenza e distribuzione degli errori di predizione, calcolati come (valore predetto - valore reale). Il picco sul valore 0 indica che il modello prevede correttamente nella maggioranza dei casi, nonostante degli errori chiaramente evincibili, perlopiù al conteggio in eccesso.

Demo

È stata implementata una demo in Python usando la libreria Streamlit, la quale consente di creare interfacce web interattive, dove si è inserita una zona per il caricamento di immagini e subito sotto la predizione del numero di individui. In particolare, l'interfaccia consente di confrontare il valore stimato con quello reale, essendo presenti due aree nella stessa pagina, una per la predizione ed un'altra per l'etichetta di ground truth, estratta dal nome del file se questo è nel formato previsto, discusso precedentemente nel paragrafo dataset. Se il nome del file non è nel formato previsto, si manifesta un messaggio d'errore, ma le predizioni vengono mostrate comunque.

Codice

Il progetto è organizzato nella cartella `progetto_ML\`, contenente:

- `codice\`: la cartella principale, all'interno si hanno i tre file sorgente `contapersone_resnet50.py`, che è il codice per il train del modello, `valutazione_test.py`, il quale invece è il codice per provare il modello sul test set, infine `demo.py`, che è lo script per la demo. Oltre a questi, è presente il file `best_ResNet50_crowd.pth` che contiene il modello finale. NB. Se la fase di train viene ripetuta, il file del modello viene sovrascritto;
- `dataset\`: cartella che contiene il dataset di train e validation, essa è divisa in `train\file_jpg` e `train\file_npy`, le quali contengono gli stessi dati ma in formati differenti;
- `relazione\`: questa cartella contiene la relazione di quanto svolto, in formato .docx e .pdf, oltre ad un video dimostrativo della demo;
- `test_dataset\`: Infine, questa cartella contiene le immagini di test.

Le istruzioni per il corretto funzionamento dei codici sono presenti, sotto forma di commenti, negli header dei codici stessi. Questi ultimi sono inoltre commentati puntualmente in ogni riga ritenuta necessaria, al fine di rendere chiara la funzione svolta.

Ad ogni modo, il comando di avvio da terminale è `python (nome del file sorgente).py`.

Conclusioni

Il modello sviluppato è in grado di stimare il numero di persone presenti in immagini con folla ottenendo un MAE medio di circa 4.16 sul test set. Nonostante la buona generalizzazione, persistono difficoltà nei casi in cui il contesto, o le condizioni di luce, delle immagini di test siano molto diverse da quello del training set. Miglioramenti futuri potrebbero includere l'utilizzo di architetture più recenti e potenti (come EfficientNet o Vision Transformers), e/o una maggiore quantità di dati di addestramento, ottenuti in situazioni più varie.

Tale progetto è stato molto utile per consolidare quanto appreso dallo studio della teoria e comprendere meglio il funzionamento di una CNN e la sua implementazione. Inoltre, mi ha permesso di confrontarmi con i tipici problemi legati all'addestramento di una rete neurale convoluzionale, quali l'overfitting, la scelta degli iperparametri e del dataset, senza dimenticare quanto appreso in generale sul linguaggio Python.